Mini SOC Environment Project

Network Setup | Firewall Analysis | Traffic Investigation

Name: Tarikur Rahman Seum

Executive Summary

This project builds a small, functional Security Operations Centre (SOC) environment using three virtual machines: an internal workstation, a gateway firewall, and a DMZ web server. The goal was to simulate how a segmented network operates, how traffic flows between zones, and how security controls such as firewall rules, logging, and packet inspection help protect the environment. The gateway was configured to perform routing, NAT, and selective firewall filtering, allowing only authorised traffic to pass between the internal network, DMZ and the internet.

The DMZ server hosted an Apache web application, enabling analysis of HTTP behaviour through both server-side logs and packet captures. Wireshark was used to inspect GET and POST login submissions, demonstrating how insecure protocols expose credentials in plaintext. Logs from the web server further confirmed how requests appear at the application layer and how different methods handle sensitive data.

Overall, this project provides a practical demonstration of core SOC skills, including network segmentation, firewall administration, traffic analysis and identification of insecure communication patterns. It highlights how proper network design and monitoring can detect weaknesses and support stronger security controls.

Background

This project creates a small security environment that acts like a simplified SOC. It includes an internal workstation, a gateway that manages routing and firewall rules, and a web server in a DMZ. The setup allows me to observe how traffic moves across network zones, how the firewall responds to different activity, and what information is recorded in system and application logs. It provides a controlled space to develop practical skills used in real security operations. This environment mirrors common SOC workflows by combining network segmentation, traffic monitoring, and log analysis to identify insecure communication patterns.

Machines Used

- **Gateway Server (Ubuntu Server)**
  Handles routing, NAT, firewall rules and logging across the three network zones.
- **Internal Workstation (Kali Linux)**
  Generates network traffic, performs connectivity tests and captures packets.
- **DMZ Web Server (Ubuntu with Apache)**
  Hosts web pages and produces access logs used for analysing HTTP behaviour.

Project Outcomes

- A functioning three-zone network with clear separation
- A firewall configured with allow and block rules
- Logged traffic showing ICMP, DNS, HTTP and login activity
- Packet captures of insecure HTTP credentials
- Clear findings and practical security recommendations

Skills Demonstrated

- Traffic and log analysis
- Identifying insecure patterns in network activity
- Linux system configuration
- Routing, NAT and iptables firewall management
- Apache web server administration
- Packet capture and filtering in Wireshark

Project Objective

This project demonstrates the design and implementation of a segmented Mini SOC environment using VirtualBox. The environment models a real-world enterprise network by separating systems into an Internal Network, a DMZ, and an Internet-facing gateway protected by firewall controls. The objective of this project is to analyse network traffic flow, enforce security policies using firewall rules, and examine how insecure web application behaviour can be detected through system logs and packet-level analysis. This environment reflects the responsibilities of a SOC analyst by combining network segmentation, traffic monitoring, and security visibility into a single controlled lab-based deployment.

Section 1: Network Setup and Routing

Overview

This section sets up the network environment used throughout the project. It includes an internal workstation, a gateway that processes and forwards traffic, and a DMZ network that will host a web server later. The goal is to create a controlled space where network flow, routing, and firewall behaviour can be observed and analysed.

Step 1 – Analyse the Network Topology

We know that one of the core principles in network security is to segment a network into zones that separate trusted and untrusted areas. Traffic between these zones is controlled through defined choke points, which makes it easier to monitor and enforce security policies. A simple logical topology is shown below:

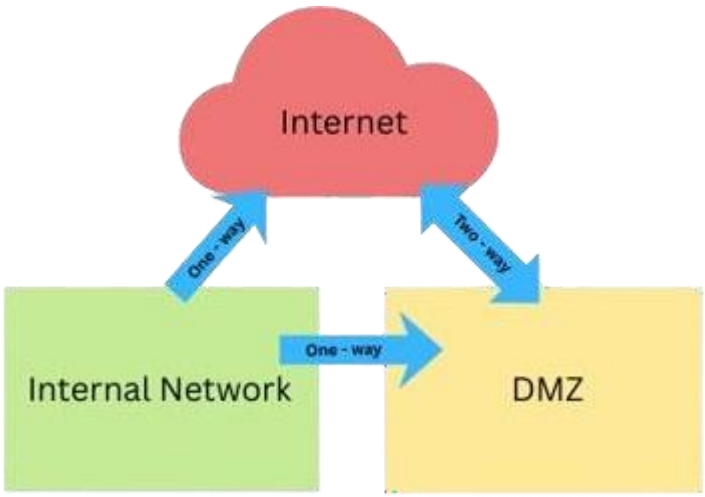


Figure 1 – Simple zone diagram showing intended traffic flow

This structure separates the Internal Network from the DMZ and restricts how traffic flows between them and the Internet.

VirtualBox is used to create this environment by running multiple virtual machines connected to different virtual networks. Each virtual network represents a zone with

its own security level and purpose. A more detailed view of the target setup is shown below:

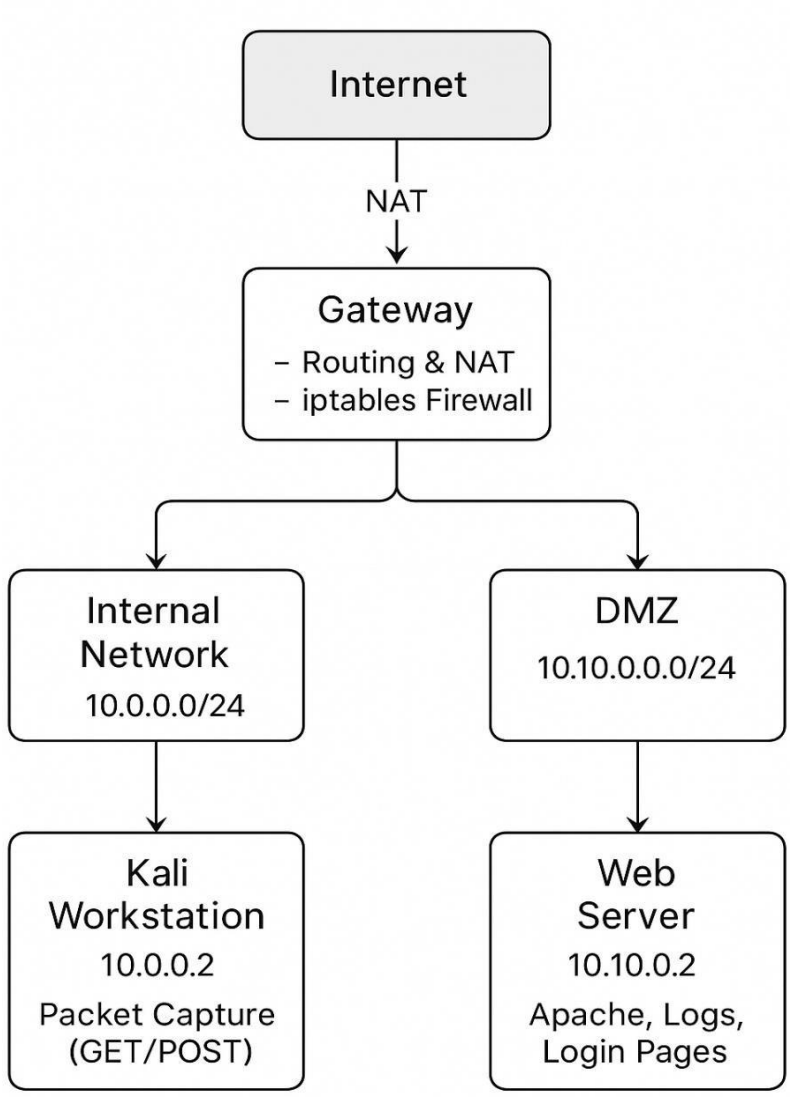


Figure 2 - Network Topology Diagram

This diagram shows the complete three-zone SOC environment: the Internal Network, the Gateway with routing and NAT, and the DMZ web server. It illustrates the traffic flow, segmentation, and IP addressing used throughout the project.

Step 1.1 – Gateway Setup

The gateway is the core component of this environment. It connects all network zones and controls how traffic moves between them. In this project, the gateway is responsible for routing packets, performing Network Address Translation (NAT), enforcing firewall rules, and logging forwarded traffic for later analysis.

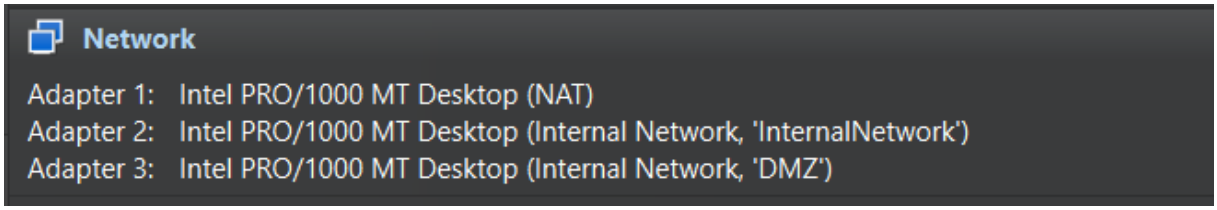
To build this component, an Ubuntu Server virtual machine was deployed and configured with multiple network interfaces so it could connect to the Internet, the Internal Network, and the DMZ at the same time.

Gateway Virtual Machine Configuration

The gateway system was created using a pre-configured Ubuntu Server image to ensure a stable and consistent base for network and firewall configuration. Once imported into VirtualBox, the machine was named **gateway** and started for initial setup.

To support communication across all zones, the gateway was configured with three network adapters:

- A NAT adapter to provide Internet connectivity
- An Internal Network adapter for the Internal Network zone
- An Internal Network adapter for the DMZ zone



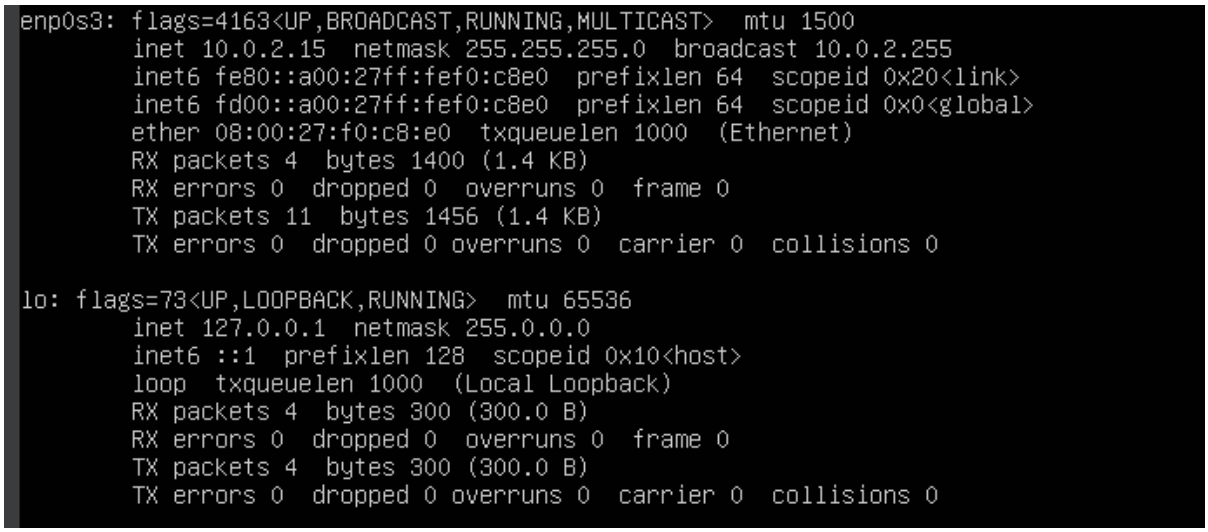
Screenshot 1 – Gateway virtual machine network adapters

This screenshot 1 shows the gateway configured with three network interfaces, allowing it to act as the central routing and control point between all network zones.

Initial Network State

Before assigning IP addresses to the Internal Network and DMZ interfaces, the gateway's network state was examined to understand its default configuration.

Running the ifconfig command showed that only the NAT interface and the loopback interface were active at this stage. The additional interfaces were visible but had no IP addresses assigned yet, which is expected for a newly imported system.



Screenshot 2 – Gateway interface status before configuration.

This screenshot 2 confirms that only the NAT interface is active, while the Internal Network and DMZ interfaces are present but not yet configured.

The routing table was then checked using the ip route command. At this point, the gateway only had a default route through the NAT interface, meaning it could reach the Internet but was not yet routing traffic for the internal or DMZ networks.

Screenshot 3 – Initial routing table on the gateway

This screenshot 3 shows the default route through the NAT interface, with no routes defined for the Internal Network or DMZ.

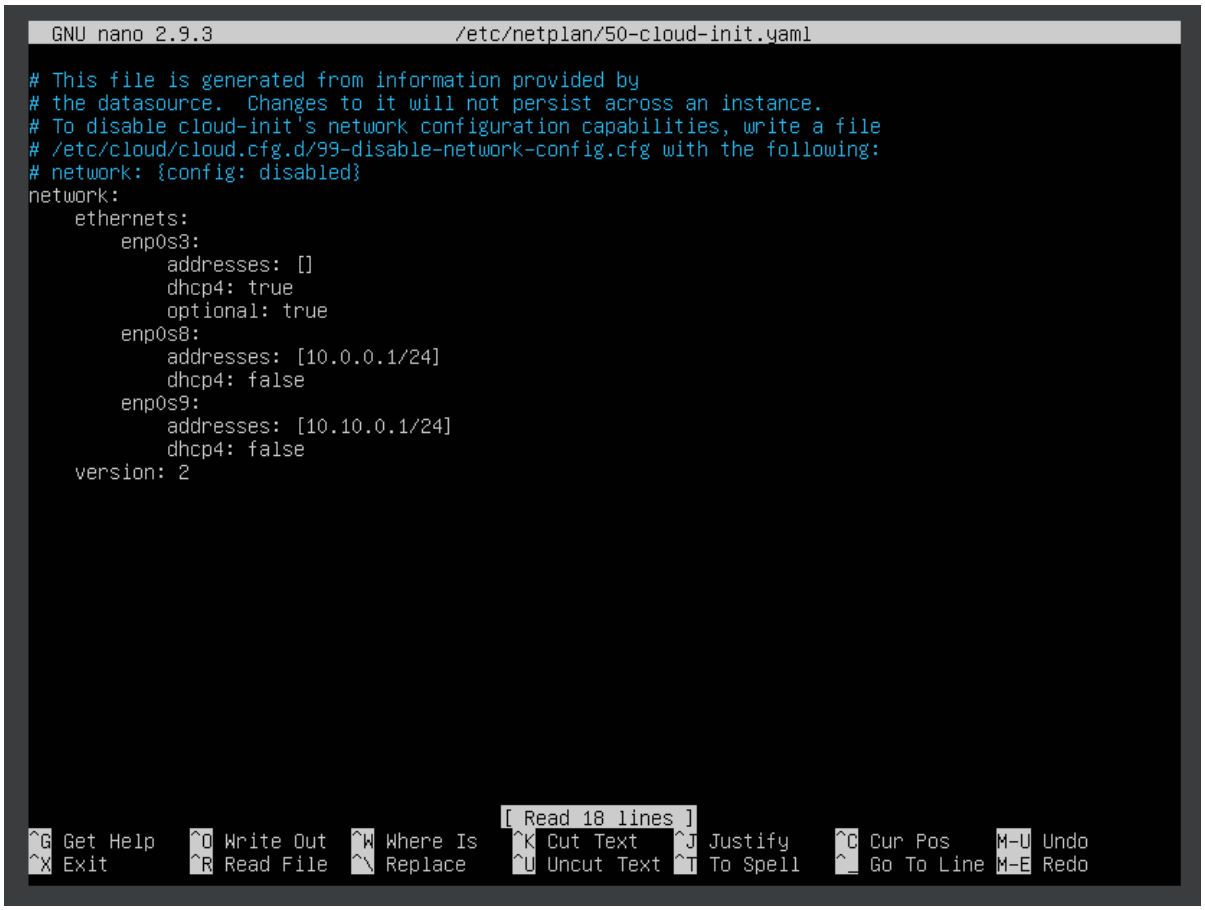
Assigning Network Addresses

To allow the gateway to communicate with both zones, static IP addresses were assigned to the Internal Network and DMZ interfaces by editing the netplan configuration file located at /etc/netplan/50-cloud-init.yaml.

The following addresses were used:

- Internal Network interface: **10.0.0.1/24**
- DMZ interface: **10.10.0.1/24**

These addresses were chosen to match the planned network design and to provide clear separation between the two zones.



Screenshot 4 – Netplan configuration with static IP addresses

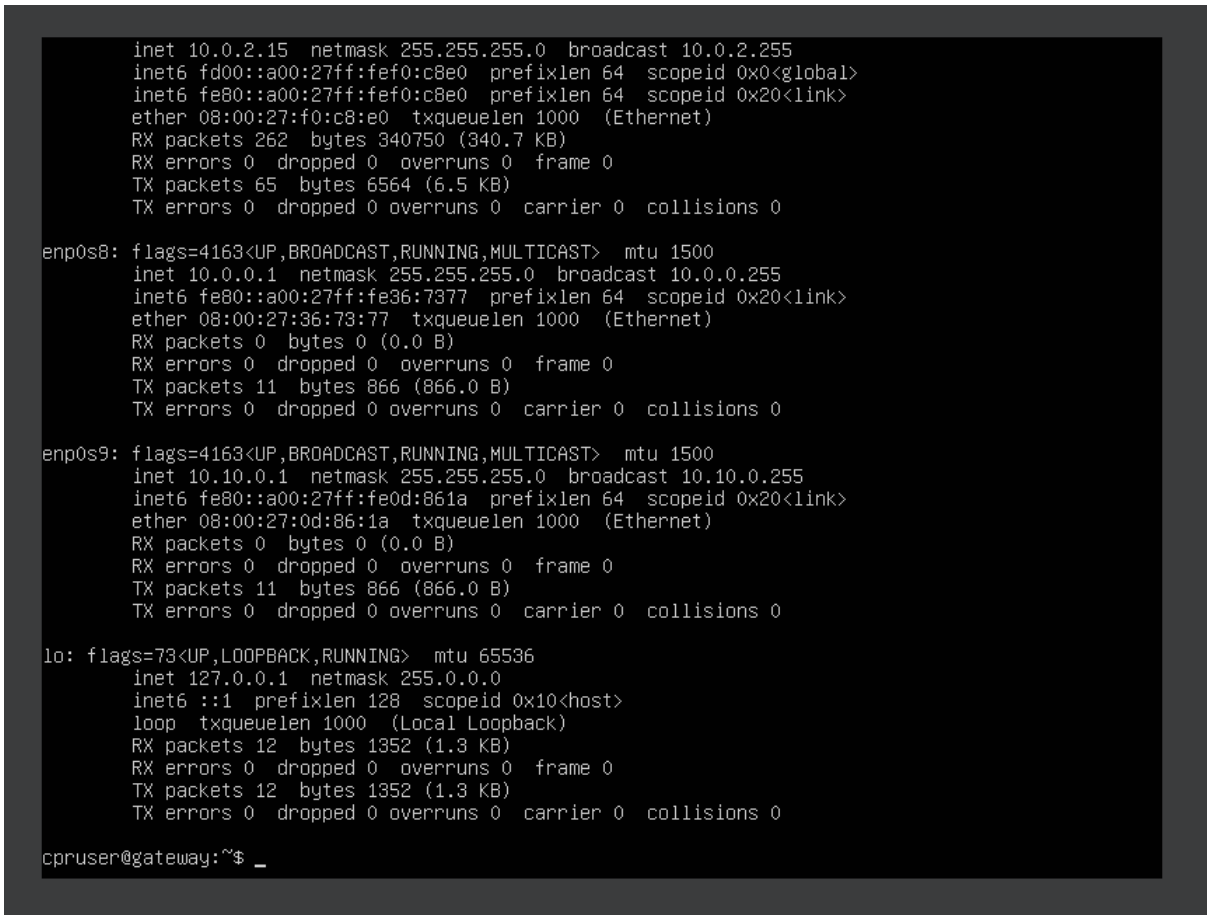
This screenshot 4 shows the updated netplan configuration after assigning static IP addresses to the Internal Network and DMZ interfaces.

After saving the changes, the new configuration was applied using the command: sudo netplan apply

Verifying Gateway Connectivity

Once the configuration was applied, the network interfaces were checked again to confirm that all settings were active.

Running ifconfig showed that all three interfaces were now operational, each with the correct IP address. The gateway was now connected to the Internal Network, the DMZ, and the Internet at the same time.



Screenshot 5 – Gateway interfaces after configuration

This screenshot confirms that the gateway is fully configured, with active interfaces for all network zones. At this point, the gateway is ready to perform routing and firewall enforcement for the rest of the project.

This completes the configuration of the gateway’s network interfaces and prepares the system for routing tests and the client setup in the next section.

Step 1.2 – Internal Workstation Setup (Kali Linux)

The internal workstation represents a typical user system inside the protected network. In this project, a Kali Linux virtual machine is used as the internal client because it provides built-in tools for network testing and packet capture. This system is responsible for generating traffic, testing connectivity across zones, and capturing network packets for later analysis.

Kali Virtual Machine Deployment

The internal workstation was created using a pre-configured Kali Linux OVA file. Using a prepared image ensures consistency and provides the necessary tools for traffic inspection without additional setup.

After importing the image into VirtualBox, the virtual machine was renamed to **kaliclient** and started for configuration. At this stage, the system was prepared to join the Internal Network zone.

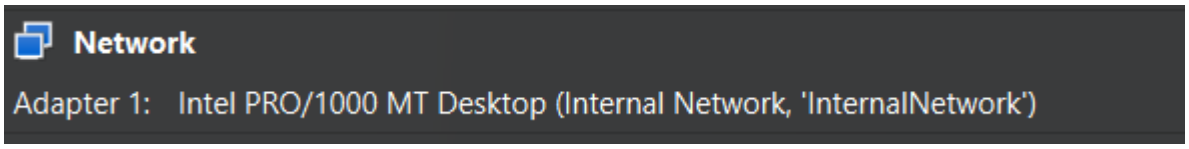
Network Adapter Configuration

The Kali client only requires a single network interface, as it should not directly access the Internet or the DMZ. Instead, all external communication must pass through the gateway.

The network adapter was configured as follows:

- Adapter 1: Internal Network (InternalNetwork)

This setup ensures that the Kali client is fully isolated within the Internal Network zone and can only communicate with the gateway.



Screenshot 6 – Kali virtual machine network adapter configuration

This screenshot 6 shows the Kali client connected to the Internal Network, ensuring that all traffic is forced through the gateway.

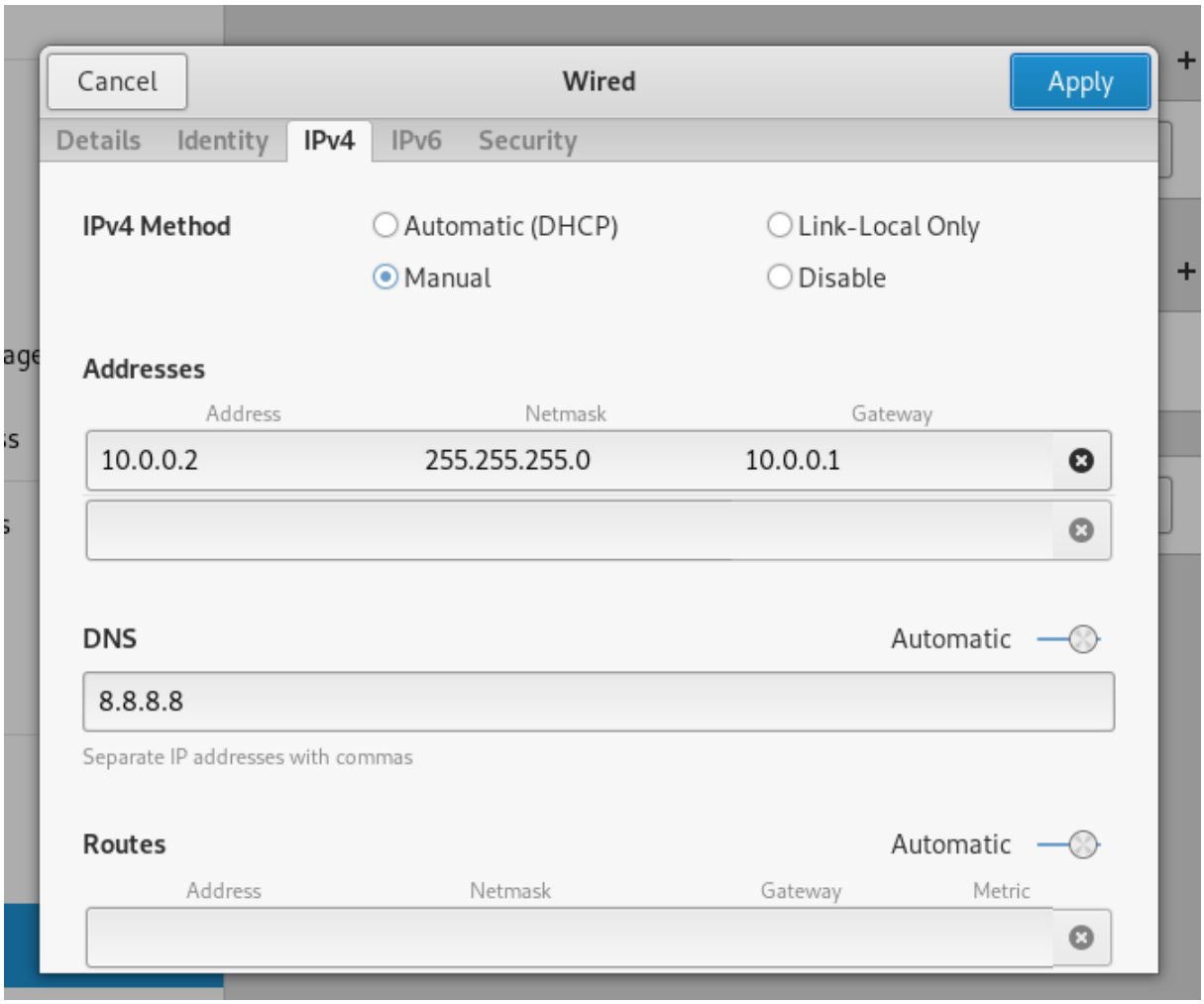
Configure Static IP via GUI

The Kali client was configured with a static network address to ensure reliable communication within the Internal Network zone and to define its path (Gateway) to external networks.

The Kali VM uses a single interface connected to the Internal Network zone. It must be configured with a static IP address in the 10.0.0.x range to communicate with the Gateway (10.0.0.1).

Parameter	Configuration	Technical Rationale

IPv4 Method	Manual	To enforce a fixed IP address, which is crucial for firewall rules and reliable routing within a static lab environment.
IP Address	10.0.0.2	A unique host address within the Internal Network subnet (10.0.0.0/24).
Netmask	255.255.255.0	Defines the local network scope (/24).
Gateway	10.0.0.1	The address of the Gateway Server's Internal Network interface (enp0s8), which serves as the next-hop router for all non-local traffic[4].
DNS	8.8.8.8	Manually specified Google Public DNS to ensure name resolution can occur once firewall rules are implemented.



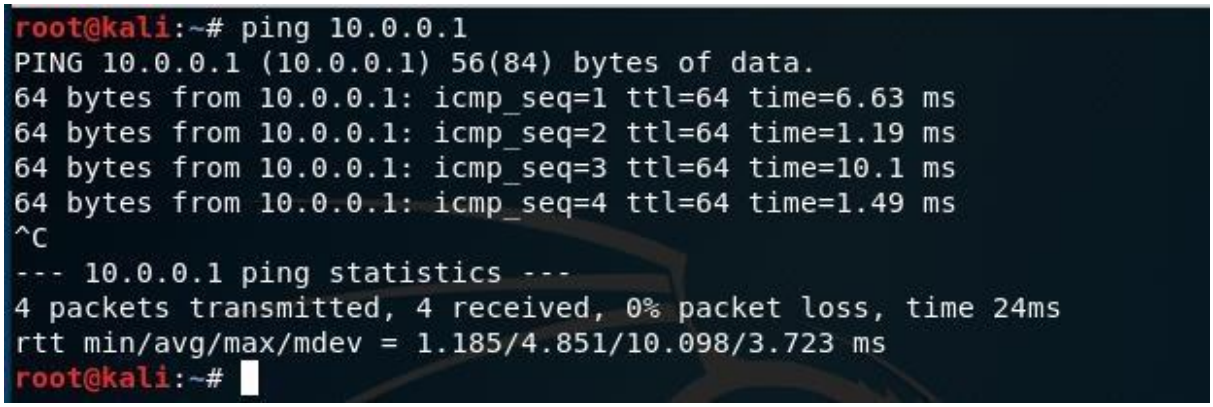
Screenshot 7 - Kali Client Static IP Configuration

From the Screenshot 7 we can see the Kali Network Manager GUI showing the Manual IPv4 settings (Address, Netmask, Gateway, and DNS) applied, confirming the client's static identity in the Internal Network.

Verification of Internal Connectivity

After configuring the network settings, connectivity between the Kali client and the gateway was tested to confirm that the Internal Network was functioning correctly.

A ping test was performed from the Kali client to the gateway's Internal Network interface (**10.0.0.1**). Successful responses confirmed that the client could reach the gateway and that the internal network configuration was correct.



Screenshot 8 – Successful connectivity test between Kali and gateway

This screenshot 8 confirms that the Kali client can communicate with the gateway, validating the internal network setup.

Step 1.3: Configuring the Gateway as a Router

At this stage, the gateway is connected to all network zones, but it is not yet forwarding traffic between them. To allow the Internal Network to communicate with external networks, the gateway must be configured to perform packet forwarding and Network Address Translation (NAT). This step transforms the gateway from a simple multi-homed system into an active routing device.

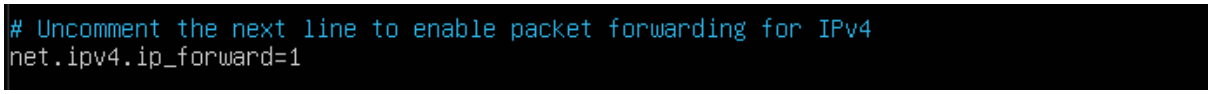
Enabling IP Packet Forwarding

By default, Linux does not forward packets between interfaces. To enable routing functionality, IPv4 packet forwarding was activated at the kernel level.

This was done by editing the system configuration file and enabling the appropriate setting:

- The file /etc/sysctl.conf was opened for editing
- The parameter net.ipv4.ip_forward was enabled

Once applied, this change allows the gateway to forward packets between the Internal Network, DMZ, and Internet interfaces.



Screenshot 9 – Enabling IPv4 Packet Forwarding in sysctl.conf

This screenshot 9 shows the packet forwarding setting enabled in the system configuration, confirming that the gateway is permitted to route traffic between interfaces.

Configure NAT and Forwarding in iptables

Next, the iptables firewall is configured to handle Network Address Translation (NAT) and allow the traffic flow into the forwarding chain.

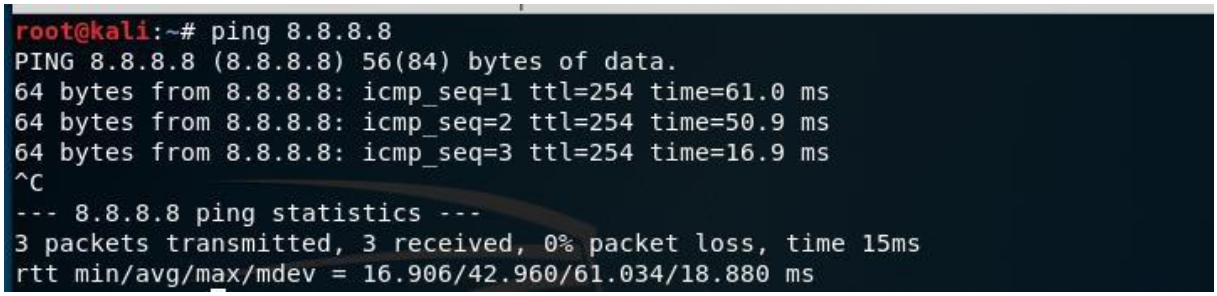
Commands to Run on Gateway Server:

$ sudo modprobe iptable_nat	Load the NAT kernel module for iptables
$ sudo iptables -t nat -A POSTROUTING -o enp0s3 -j MASQUERADE	Tell iptables to NAT any traffic exiting via the interface enp0s3
$ sudo iptables -A FORWARD -i enp0s8 -j LOG --log-level 6	Tell iptables to forward any traffic received on interface enp0s8

Verification of External Connectivity

After enabling routing and NAT, external connectivity was tested from the Kali client to confirm that the gateway was functioning correctly.

A ping test to an external IP address was performed from the Internal Network. Successful responses confirmed that traffic was being forwarded through the gateway and that NAT was applied correctly.



Screenshot 10: Successful External Ping Test (Post-Routing Configuration)

This screenshot 10 shows successful ping replies from an external address, confirming that the gateway is now operating as a router and allowing outbound traffic from the Internal Network.

SECTION 2: Firewall Analysis

Overview

In Section 1, the gateway was configured to route traffic between network zones and provide Internet access to the Internal Network. While this setup enables connectivity, it does not enforce any security controls. In this section, the focus shifts from basic routing to security enforcement by configuring the gateway firewall.

Firewall rules are introduced progressively to control which types of traffic are allowed to pass between zones and which are blocked. By observing how traffic is permitted, denied, and logged, this section demonstrates how a gateway transitions from a simple router into an active security control point, supporting SOC-style monitoring and analysis.

Step 2.1 – Resetting and Inspecting the Firewall

To begin the firewall configuration process, the existing iptables rules on the gateway were cleared to establish a known baseline. This ensures that any changes in network behaviour observed in later steps are directly caused by the rules introduced in this section.

After flushing the firewall rules, all chains returned to their default state, allowing traffic to pass freely between interfaces. This baseline highlights the difference between an operational but unsecured network and one that actively enforces security policy.

Screenshot 11 – iptables after flushing rules

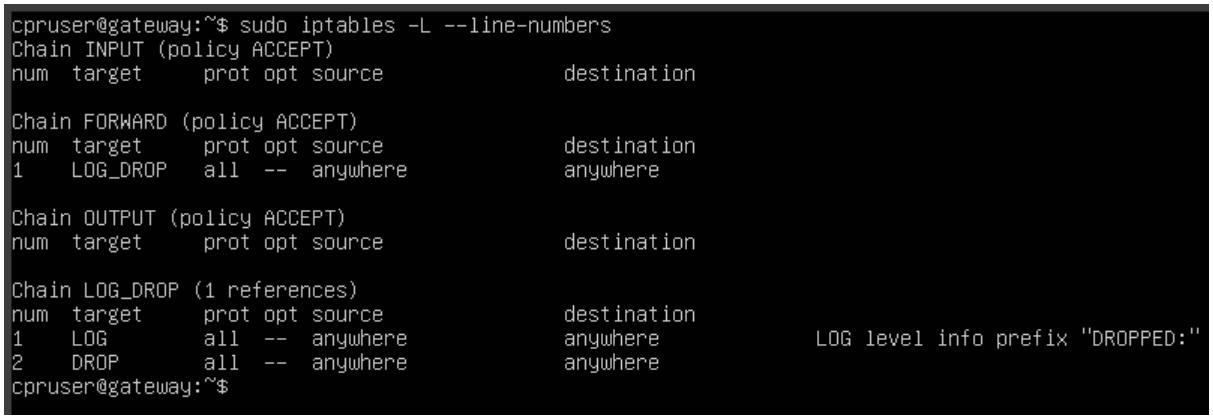
This screenshot shows the firewall state immediately after all rules were cleared. The FORWARD chain is set to its default ACCEPT policy, confirming that no filtering or restriction is currently applied.

Step 2.2 – Creating the LOG_DROP Chain

A key part of firewall design is to block everything by default and only allow traffic that is explicitly needed. To achieve this, a custom chain was created named *LOG_DROP*, which logs traffic before dropping it.

Commands Run on Gateway:

- sudo iptables -N LOG_DROP
- sudo iptables -A LOG_DROP -j LOG --log-prefix "DROPPED:" --log-level 6
- sudo iptables -A LOG_DROP -j DROP
- sudo iptables -A FORWARD -j LOG_DROP



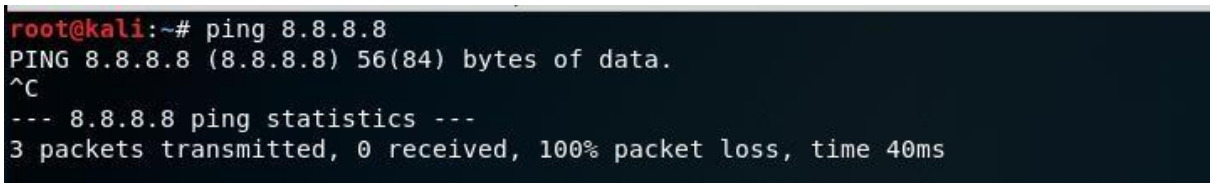
Screenshot 12 – LOG_DROP Chain in iptables

From the Screenshot 12, iptables output showing the custom LOG_DROP chain with logging and drop rules, and the FORWARD chain configured to send all traffic into this chain.

Step 2.3 – Testing the Block-All Rule

At this stage, all forwarding traffic between zones is blocked. The Kali client should no longer be able to reach the Internet.

Command Run on Kali: ping 8.8.8.8



Screenshot 13 – Blocked ICMP Traffic

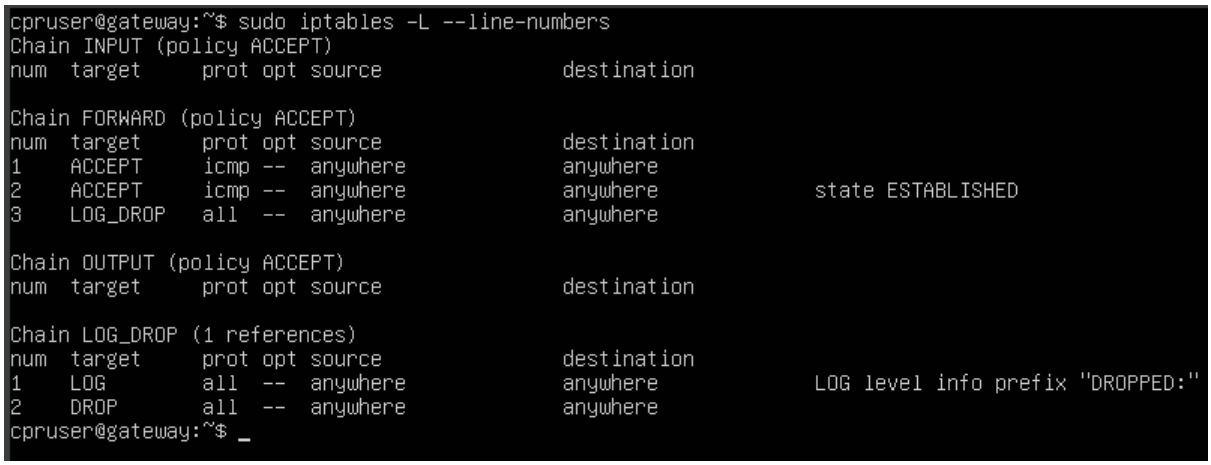
From the Screenshot 13, Kali client's failed attempt to ping an external IP, confirming that all forwarding traffic is being blocked by the gateway's default LOG_DROP rule.

Step 2.4 – Allowing ICMP (Ping) Traffic to the Internet

To restore basic connectivity, a rule was added to allow ICMP traffic from the Internal Network (enp0s8) to the Internet. A second rule allows the return packets as part of an established connection.

Commands Run on Gateway:

- sudo iptables -I FORWARD 1 -i enp0s8 -p icmp -j ACCEPT
- sudo iptables -I FORWARD 2 -i enp0s3 -p icmp -m state --state ESTABLISHED -j ACCEPT



Screenshot 14 – ICMP Rules Inserted

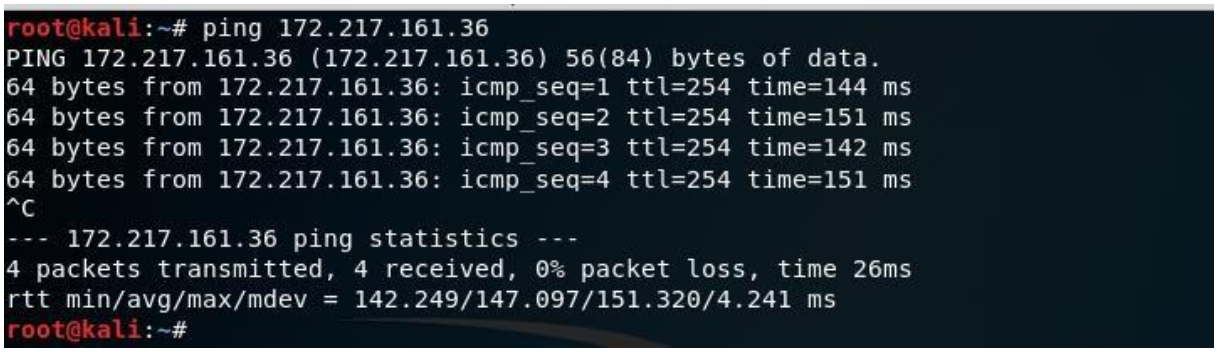
From the Screenshot 14, The FORWARD chain displaying newly inserted ICMP accept rules that allow outbound echo requests and inbound echo replies for established connections.

Step 2.5 – Testing ICMP Traffic

With ICMP allowed, the Kali client can now successfully ping an external IP address. Pinging a domain name will still fail because DNS has not been allowed yet.

Command Run on Kali:

- ping 172.217.161.36



Screenshot 15 – Successful ICMP Connectivity (IP Only)

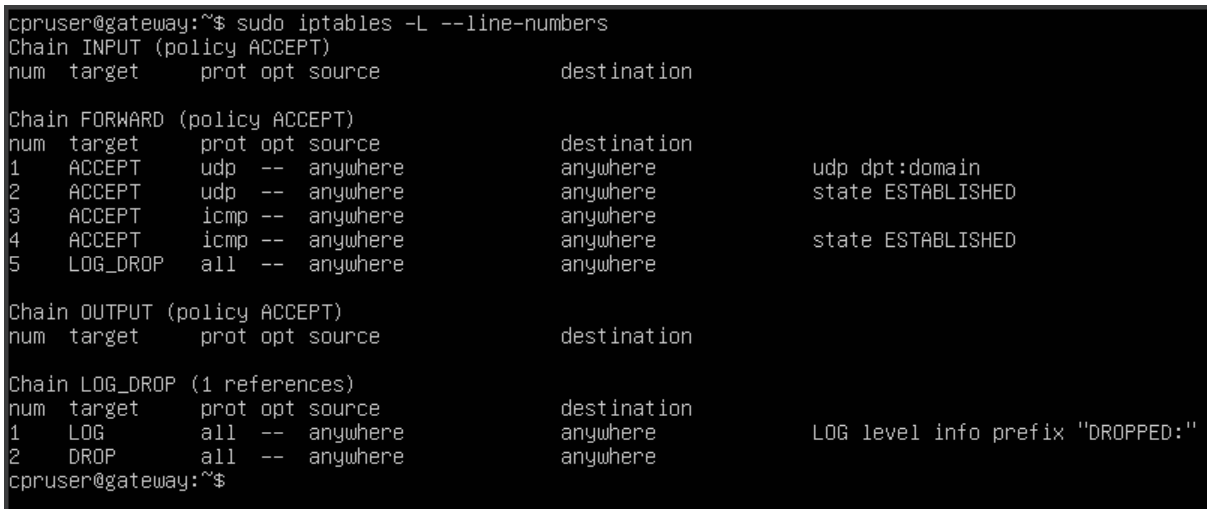
From the Screenshot 15, Kali client successfully pinging an external IP address after ICMP rules were added. DNS-based ping is not yet possible because DNS rules have not been applied.

Step 2.6 – Allowing DNS Traffic

Ping by hostname still may fail before DNS rules are added. To enable DNS resolution, UDP traffic to port 53 must be allowed.

Commands Run on Gateway:

- sudo iptables -I FORWARD 1 -i enp0s8 -p udp --dport 53 -j ACCEPT
- sudo iptables -I FORWARD 2 -i enp0s3 -p udp -m state --state ESTABLISHED -j ACCEPT



Screenshot 16 – DNS Rules Added

From the Screenshot 16, Inbound DNS rule allowing UDP port 53 traffic from the Internal Network, with a matching ESTABLISHED rule to permit DNS replies.

Step 2.7 – Testing DNS Resolution

Command run on kali: ping www.google.com

```
File  Edit  View  Search  Terminal  Help
root@kali:~# ping www.google.com
PING www.google.com (142.250.124.99) 56(84) bytes of data.
64 bytes from bw-in-f99.1e100.net (142.250.124.99): icmp_seq=1 ttl=254 time=30.2 ms
64 bytes from bw-in-f99.1e100.net (142.250.124.99): icmp_seq=2 ttl=254 time=26.1 ms
64 bytes from bw-in-f99.1e100.net (142.250.124.99): icmp_seq=3 ttl=254 time=27.9 ms
64 bytes from bw-in-f99.1e100.net (142.250.124.99): icmp_seq=4 ttl=254 time=33.4 ms
^C
--- www.google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 10ms
rtt min/avg/max/mdev = 26.105/29.388/33.351/2.705 ms
root@kali:~#
```

Screenshot 17 – Successful DNS-Based Ping

From the screenshot 17 we can see that A successful ping to www.google.com confirming the DNS rule is working and name resolution is now permitted.

Step 2.8 – Allowing Web Traffic (HTTP/HTTPS)

To enable web browsing from the Internal Network to the Internet, rules were added for TCP ports 80 and 443.

Commands Run on Gateway:

- sudo iptables -I FORWARD 5 -i enp0s8 -p tcp --dport 80 -j ACCEPT
- sudo iptables -I FORWARD 5 -i enp0s8 -p tcp --dport 443 -j ACCEPT

- sudo iptables -I FORWARD 7 -i enp0s3 -p tcp --sport 80 -m state --state ESTABLISHED -j ACCEPT
- sudo iptables -I FORWARD 7 -i enp0s3 -p tcp --sport 443 -m state --state ESTABLISHED -j ACCEPT

```
cpruser@gateway:~$ sudo iptables -L --line-numbers
Chain INPUT (policy ACCEPT)
num  target     prot opt source               destination

Chain FORWARD (policy ACCEPT)
num  target     prot opt source               destination
1    ACCEPT     udp  --  anywhere             anywhere             udp dpt:domain
2    ACCEPT     udp  --  anywhere             anywhere             state ESTABLISHED
3    ACCEPT     icmp --  anywhere             anywhere
4    ACCEPT     icmp --  anywhere             anywhere             state ESTABLISHED
5    ACCEPT     tcp  --  anywhere             anywhere             tcp dpt:https
6    ACCEPT     tcp  --  anywhere             anywhere             tcp dpt:http
7    ACCEPT     tcp  --  anywhere             anywhere             tcp spt:https state ESTABLISHED
8    ACCEPT     tcp  --  anywhere             anywhere             tcp spt:http state ESTABLISHED
9    LOG_DROP   all  --  anywhere             anywhere

Chain OUTPUT (policy ACCEPT)
num  target     prot opt source               destination

Chain LOG_DROP (1 references)
num  target     prot opt source               destination
1    LOG        all  --  anywhere             anywhere             LOG level info prefix "DROPPED:"
2    DROP       all  --  anywhere             anywhere
cpruser@gateway:~$
```

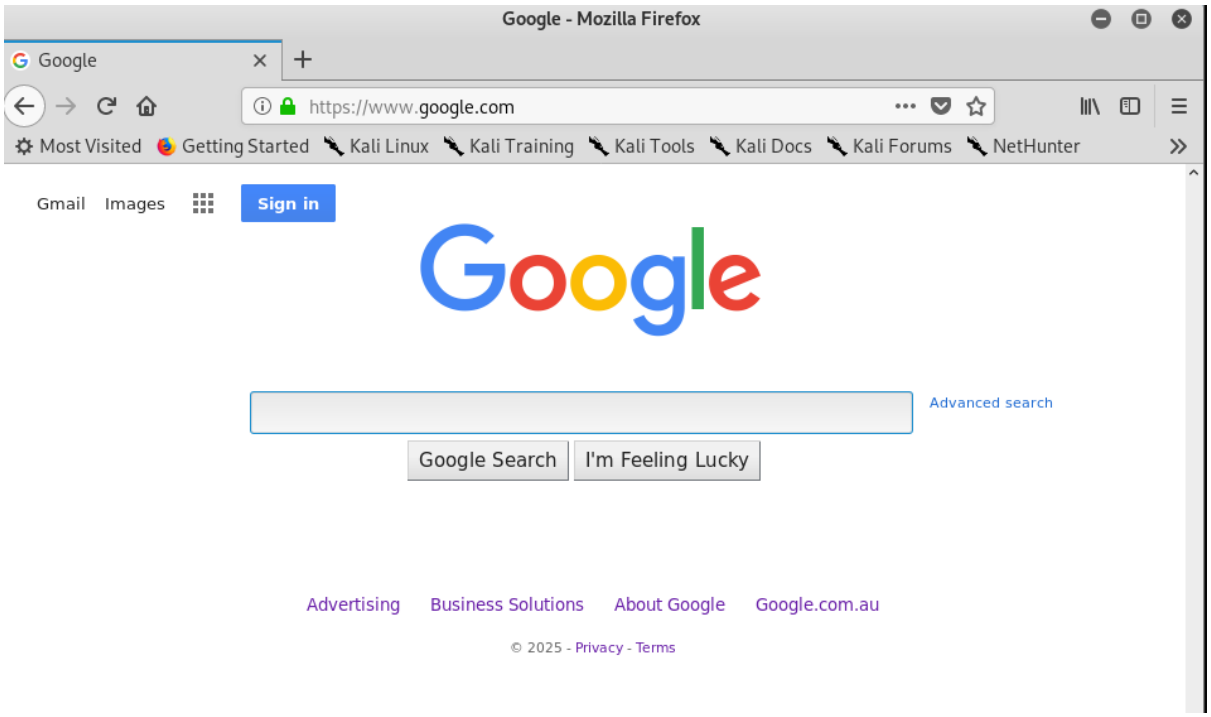
Screenshot 18 – HTTP/HTTPS Rules in iptables

From the Screenshot 18, we can see that, the FORWARD chain with newly added rules that allow outbound HTTP/HTTPS requests and inbound return traffic during established sessions.

Step 2.9 – Verifying Web Browsing from Kali

Action on Kali:

Open Firefox and browse to: https://www.google.com



Screenshot 19 – Successful Web Browsing

From the Screenshot 19 we can see, A successful HTTPS connection from the Kali client confirming that HTTP and HTTPS firewall rules are working correctly.

Firewall Policy Summary

In this section, the gateway firewall was transformed from an open routing device into a controlled security enforcement point. After clearing existing rules, a custom LOG_DROP chain was implemented to log and drop all forwarded traffic by default. Specific rules were then added incrementally to permit only required traffic types, including ICMP for connectivity testing, DNS for name resolution, and HTTP/HTTPS for web access. Return traffic was explicitly allowed using connection state tracking to ensure that only responses to legitimate outbound requests were accepted. Logging of blocked and forwarded traffic provides visibility into network activity and supports SOC-style monitoring and analysis.

SECTION 3: Web Server Deployment and Traffic Analysis

Overview

This section introduces the third virtual machine in the SOC environment: the web server hosted in the DMZ. The purpose of this component is to provide a controlled, publicly accessible service that can be monitored and analysed from a security perspective. An Apache web server is deployed, access is selectively permitted through the gateway firewall, and both server-side logs and network traffic are examined.

This setup simulates how SOC analysts investigate HTTP activity, identify insecure communication patterns, and verify that network segmentation and firewall filtering are functioning as intended.

Step 3.1 – Building the Web Server (DMZ)

A new Ubuntu Server virtual machine was deployed to act as a web server within the DMZ network. This system represents a publicly accessible service that is isolated from the internal network and protected by the gateway firewall.

The web server virtual machine was imported into VirtualBox using the provided webserver OVA template, renamed for clarity, and connected to the DMZ internal network to ensure proper segmentation.

Network Configuration

The web server requires a static IP to ensure predictable routing and firewall behaviour. The following settings were applied in /etc/netplan/50-cloud-init.yaml:

Parameter	**Value**
IP address	10.10.0.2/24
Gateway	10.10.0.1 (DMZ interface of the gateway)
Nameservers	8.8.8.8, 8.8.4.4
DHCP	Disabled

After updating the file, the configuration was applied using: sudo netplan apply

```
  GNU nano 2.9.3                    /etc/netplan/50-cloud-init.yaml

# This file is generated from information provided by
# the datasource.  Changes to it will not persist across an instance.
# To disable cloud-init's network configuration capabilities, write a file
# /etc/cloud/cloud.cfg.d/99-disable-network-config.cfg with the following:
# network: {config: disabled}
network:
    ethernets:
        enp0s3:
            addresses: [10.10.0.2/24]
            dhcp4: false
            gateway4: 10.10.0.1
            nameservers:
             addresses: [8.8.8.8, 8.8.4.4]
            optional: true
    version: 2
```

Screenshot 20 – Web Server Network Configuration

In the Screenshot 20, we can see that Static IP settings applied to the web server in the DMZ.

Step 3.2 – Testing Web Server Connectivity (DMZ → Gateway)

Connectivity tests were performed from the DMZ web server to verify how traffic flows through the gateway and to confirm that the server is correctly isolated from external networks. Three ping commands were executed: one to the gateway, one to an external IP address, and one to a domain name.

Commands run from the webserver:

- ping 10.10.0.1
- ping 172.217.161.36
- ping www.google.com

```
cpruser@webserver:~$ ping 10.10.0.1
PING 10.10.0.1 (10.10.0.1) 56(84) bytes of data.
64 bytes from 10.10.0.1: icmp_seq=1 ttl=64 time=1.19 ms
64 bytes from 10.10.0.1: icmp_seq=2 ttl=64 time=1.86 ms
64 bytes from 10.10.0.1: icmp_seq=3 ttl=64 time=1.95 ms
64 bytes from 10.10.0.1: icmp_seq=4 ttl=64 time=3.43 ms
^C
--- 10.10.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3008ms
rtt min/avg/max/mdev = 1.193/2.111/3.431/0.817 ms
cpruser@webserver:~$
cpruser@webserver:~$
cpruser@webserver:~$ ping 172.217.161.36
PING 172.217.161.36 (172.217.161.36) 56(84) bytes of data.
^C
--- 172.217.161.36 ping statistics ---
14 packets transmitted, 0 received, 100% packet loss, time 13302ms

cpruser@webserver:~$ ping www.google.com
ping: www.google.com: Temporary failure in name resolution
cpruser@webserver:~$
```

Screenshot 21 – DMZ Connectivity Test Results

From the screenshot, the ping to **10.10.0.1** was successful, confirming that the DMZ server can communicate with the gateway's DMZ interface, as both devices are located within the same subnet. The attempt to reach the external IP address **172.217.161.36** resulted in 100% packet loss because the gateway firewall does not allow outbound ICMP traffic from the DMZ interface (enp0s9), causing these packets to be dropped. The final test, pinging **www.google.com**, failed immediately due to a DNS resolution error, as DNS traffic from the DMZ is also restricted. These results demonstrate that the DMZ server can reach the gateway but is prevented from directly accessing external networks, maintaining the intended network segmentation and security controls.

Step 3.3 – Install and Configure the Apache Web Server

Apache was installed on the DMZ server so it can host web content and respond to HTTP requests coming from the internal network.

The following commands were executed on the web server:

- sudo apt update
- sudo apt install apache2
- sudo apt install php

The update command refreshes the package list, and the next two commands install the Apache web server along with PHP support. Once installation is complete, Apache starts automatically and begins listening on port 80 for incoming HTTP connections.

At this point, the service is running inside the DMZ, but it cannot be reached from the internal network yet because the gateway firewall blocks this traffic by default. Access will be enabled in the next step when the firewall rules are updated.

Step 3.4 – Allow Web Traffic from the Internal Network to the DMZ

With Apache running in the DMZ, the next task is to make the service reachable from the internal network. By default, the gateway blocks all forwarding traffic, so specific rules must be added to allow HTTP responses coming from the DMZ back to the internal client.

The following rules were added on the gateway:

- sudo iptables -I FORWARD 9 -i enp0s9 -p tcp --sport 80 -m state --state ESTABLISHED -j ACCEPT
- sudo iptables -I FORWARD 9 -i enp0s9 -p tcp --sport 443 -m state --state ESTABLISHED -j ACCEPT
- sudo iptables -I FORWARD 9 -i enp0s9 -p icmp -m state --state NEW,ESTABLISHED -j ACCEPT

- sudo iptables -I FORWARD 9 -i enp0s9 -p udp --sport 53 -m state --state ESTABLISHED -j ACCEPT

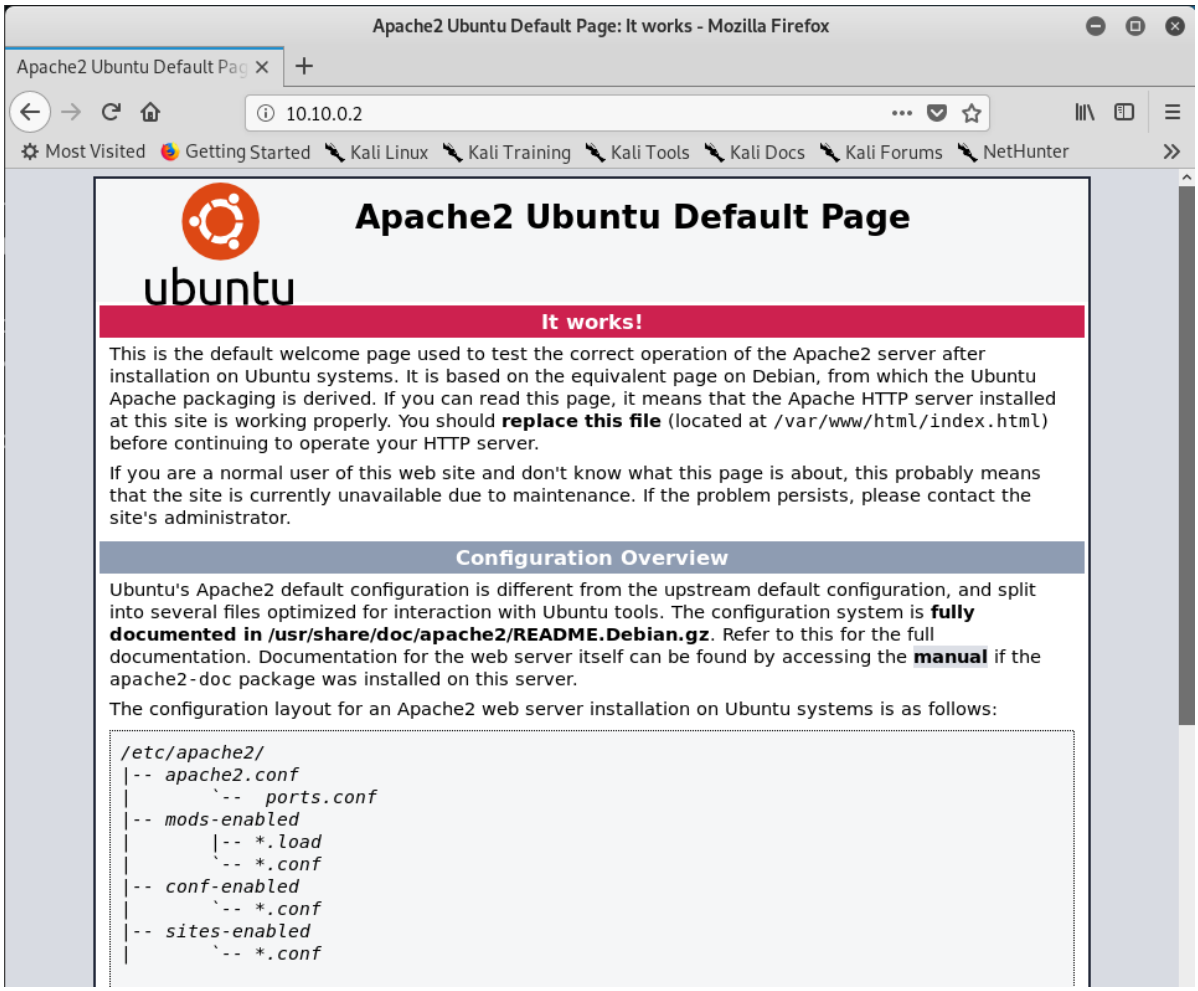
Explanation of the rules:

- Traffic entering through enp0s9 (the DMZ interface) is inspected.
- Established HTTP and HTTPS responses (ports 80 and 443) are allowed so the web server can reply to inbound requests.
- ICMP and DNS replies from the DMZ are also permitted when they belong to an established connection.
- All other traffic remains blocked, preserving the one-way access design where only the internal network is allowed to initiate connections.

After applying these rules, the internal Kali client can now access the Apache homepage by visiting:

- http://10.10.0.2

This confirms that the gateway correctly forwards allowed traffic into and out of the DMZ while still enforcing controlled segmentation.



Screenshot 22 – Apache Homepage Accessible from Internal Network

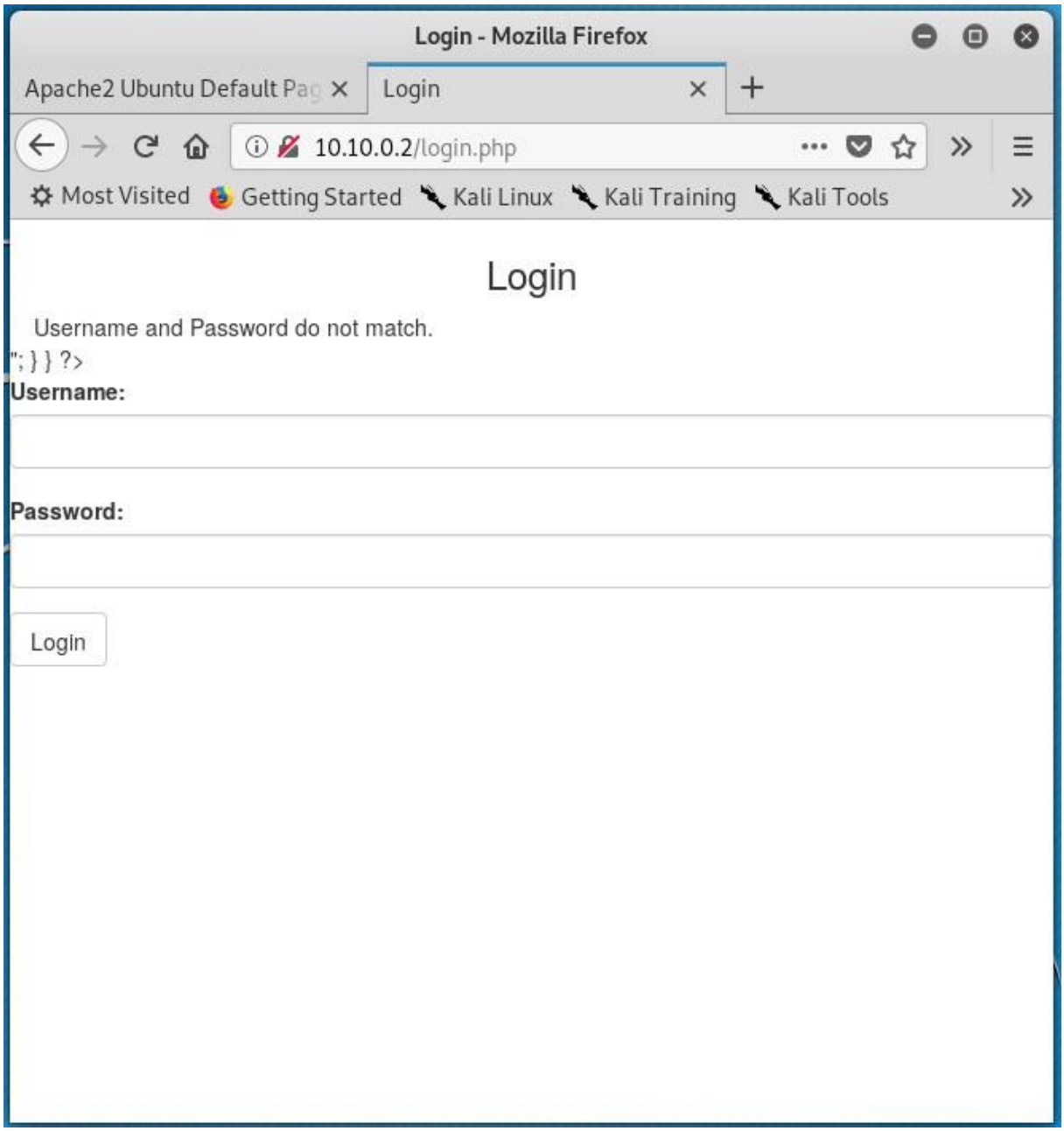
This screenshot 22 shows the Kali client successfully loading the Apache default webpage hosted in the DMZ. This confirms that the gateway firewall rules are working and HTTP traffic to the DMZ server is now permitted.

Step 3.5 – Deploy Custom Web Pages to the Web Server

To support traffic analysis and security monitoring, a set of custom web pages was added to the web server. These pages simulate basic login activity and allow the system to generate realistic HTTP requests that can be inspected later in the logs and packet captures.

A setup script included with the server image was executed to install these pages. Once completed, the new files became available through the web server and could be accessed from the internal client.

These custom pages provide controlled examples of user authentication behaviour, making it possible to observe how requests flow through the gateway, how they appear in server logs, and how sensitive information can be exposed over unencrypted HTTP connections.



Screenshot 23 – Custom Login Page Accessible from Internal Network

This screenshot 23 confirms that the custom login pages were successfully added and are now available on the web server for traffic analysis.

Step 3.6 – Inspecting Web Server Access Logs

With the login page now active, the next step was to examine how the web server records incoming traffic. Apache logs every request it receives, making the access log patterns relevant to security monitoring.

To view the most recent entries, the following command was executed on the DMZ server:

- tail -n 5 /var/log/apache2/access.log

The output shows a sequence of interactions from the internal workstation, including page loads, requests for supporting resources, and a login submission. Each entry includes details such as the client IP address, timestamp, HTTP method, requested resource, status code and browser information. These fields together provide a clear picture of how the server is being accessed and how different types of requests are processed.

This particular log sample is useful because it captures both **GET** requests (used when loading the page) and a **POST** request (generated when the login form was submitted). This distinction becomes important later when analysing how different submission methods handle user input and how they appear in logs and network captures.

Screenshot 24 – Apache Log Showing GET and POST Requests

This screenshot 24 displays recent HTTP activity recorded by Apache, including GET requests for the login page and a POST request generated by submitting the login form. These entries confirm active communication between the internal network and the DMZ web server and illustrate how different request types appear in server logs.

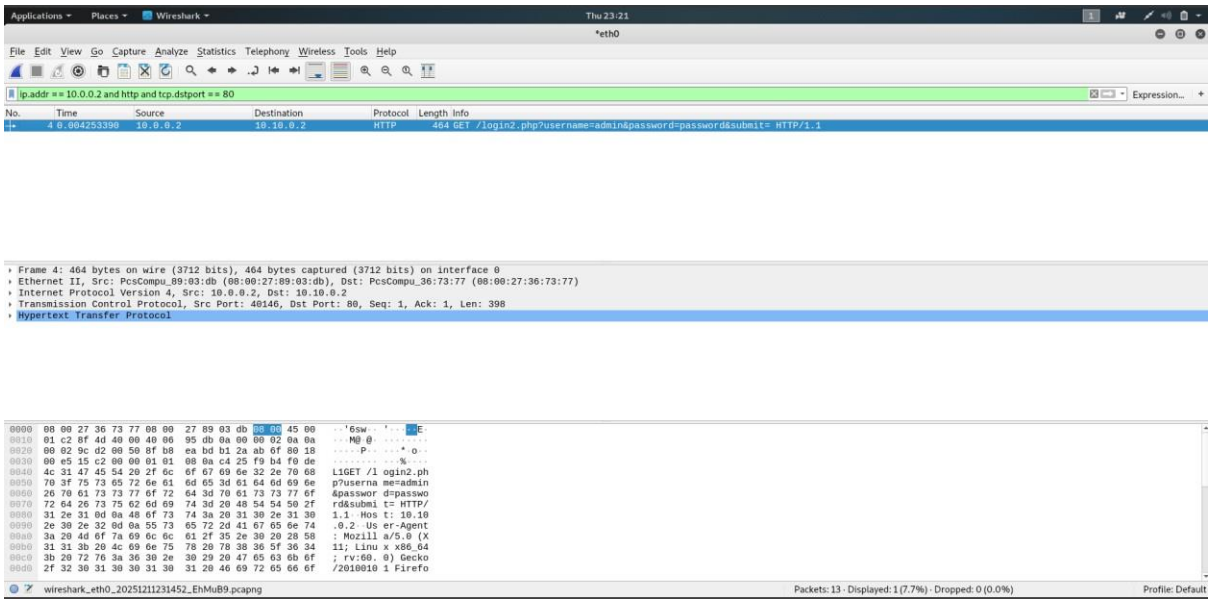
Step 3.7 – Wireshark Analysis of HTTP GET Login Traffic

Wireshark was used on the Kali workstation to capture traffic sent from the internal network to the DMZ web server. The purpose of this capture was to inspect how credentials are transmitted when using the **GET-based login page (login2.php)** and to identify any security issues in the network flow.

The following filter was applied to reduce background traffic and display only HTTP packets sent to the web server:

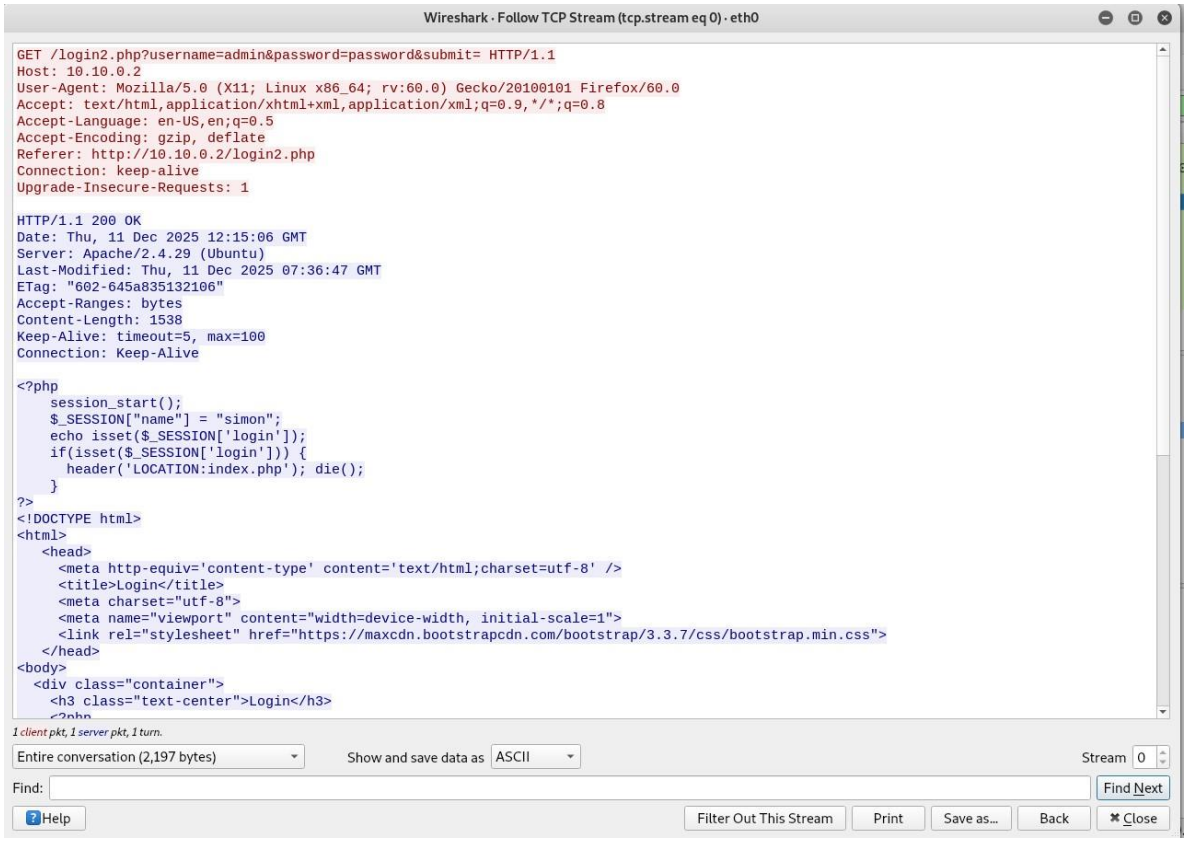
ip.addr == 10.0.0.2 and http and tcp.dstport == 80

This filter isolates HTTP traffic involving the Kali client (10.0.0.2) and ensures only port 80 packets are shown, which is the port used by Apache.



Screenshot 25 – Wireshark Capture Showing Plaintext Credentials in GET Request

This screenshot 25 shows the packet-level capture of the HTTP GET request sent from the Kali client to the DMZ web server. The request includes the username and password directly in the URL (login2.php?username=admin&password=password), confirming that the GET method exposes credentials in plaintext. The packet details and hex/ASCII view further demonstrate that the login information is transmitted without encryption, making it easily accessible to anyone monitoring network traffic.



Screenshot 26 – Full HTTP Conversation Revealing Exposed Credentials (Follow TCP Stream)

This screenshot 26 displays the full HTTP conversation obtained using Wireshark's Follow TCP Stream feature. The GET request again shows the username and password embedded in the URL, and the entire exchange between client and server is readable in plaintext. This confirms that when HTTP is used without encryption, both the request metadata and the actual content of the communication can be fully reconstructed by an attacker observing the network.

Analysis

- The GET method places form data directly in the URL, causing the username and password to appear in the request line.
- Wireshark shows the full URL containing the exposed credentials in the packet summary and header details.
- The hex/ASCII panel confirms that the login information is transmitted in plaintext over the network.
- The Follow TCP Stream view reconstructs the entire HTTP conversation, again revealing the credentials with no encryption.
- Any attacker monitoring the same network segment could capture and read the credentials with minimal effort.
- This demonstrates that GET is unsafe for authentication and should never be used for login forms, especially over unencrypted HTTP.

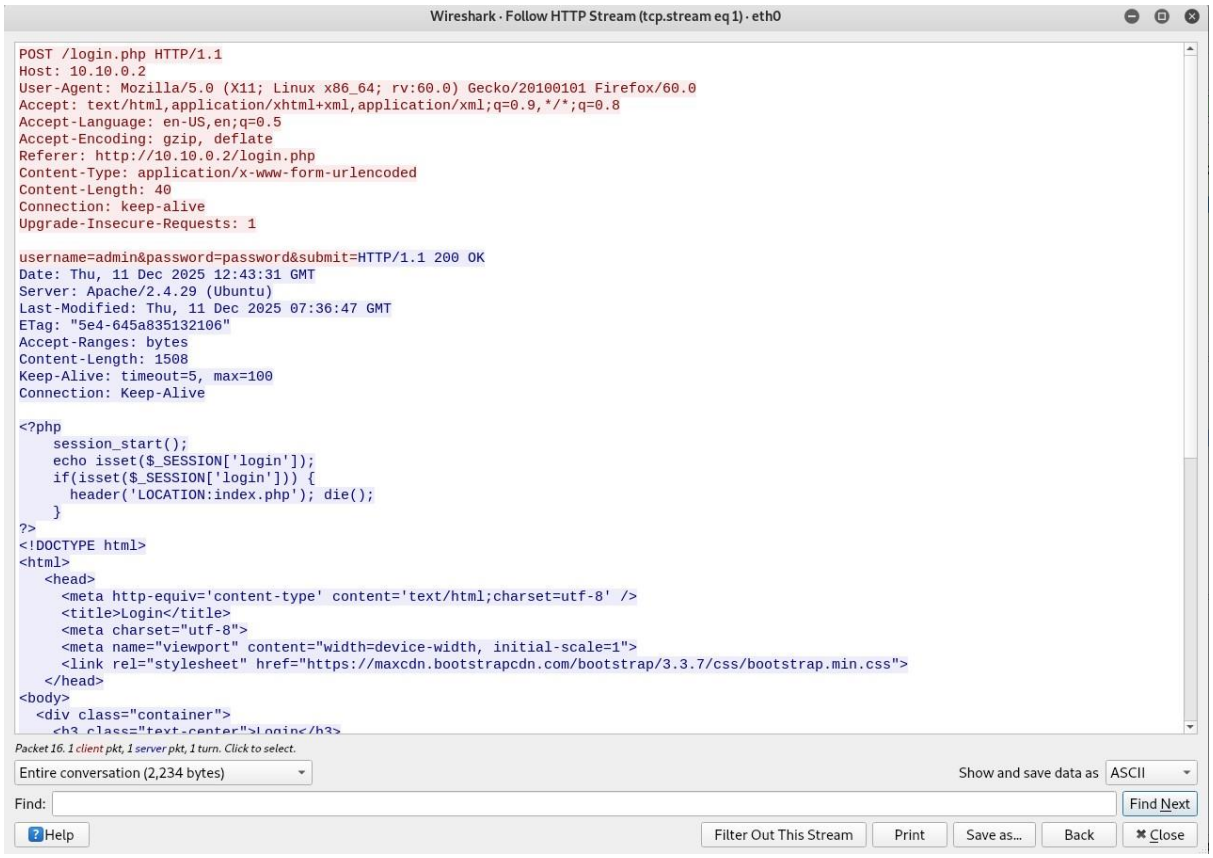
Step 3.8 – Wireshark Analysis of HTTP POST Login Traffic

To compare the security differences between GET and POST login submissions, Wireshark was used to capture the HTTP traffic generated when the form on login.php was submitted. Unlike GET, the POST method does not place credentials in the URL. Instead, the form data is included in the HTTP message body. This step examines whether this method provides any additional security when used over unencrypted HTTP.

The following filter was applied to isolate traffic between the Kali client and the DMZ web server:

ip.addr == 10.0.0.2 and http and tcp.dstport == 80

This filter displays only HTTP packets sent to port 80, which is the default port for Apache.



Screenshot 27 – Follow HTTP Stream view revealing plaintext credentials in POST request

This screenshot 27 shows the full HTTP conversation captured using Wireshark's Follow HTTP Stream feature. The request uses the POST method, which hides credentials from the URL. However, the body of the request still contains the username and password in plaintext (username=admin&password=password). Because the connection uses unencrypted HTTP, the entire request and response can be read without any decoding or decryption. This confirms that POST only prevents credentials from appearing in the URL, but does not protect them from interception on the network.

Analysis

- The POST method does not include the username and password in the URL, reducing exposure in browser history and server logs.
- Wireshark's Follow HTTP Stream output shows that the credentials are still transmitted in plaintext inside the HTTP request body.
- The packet contents can be fully reconstructed because HTTP does not provide any encryption.

- Anyone monitoring the network between the client and the DMZ server could easily capture the username and password.
- POST provides slightly better privacy than GET, but **does not provide security** when used with HTTP.
- Only HTTPS (TLS encryption) can protect the confidentiality of login credentials during transmission.

Security Recommendations

Based on the observations made throughout this project, the following security improvements are recommended:

1. Implement HTTPS for all login and sensitive pages
Using HTTP exposes credentials in plaintext during transmission. Enabling HTTPS (TLS encryption) would protect authentication data and prevent credential theft.

2. Restrict outbound traffic from the DMZ
The DMZ server should only send traffic required for essential operations. Blocking unnecessary outbound connections reduces the attack surface and limits lateral movement if the server is compromised.

3. Strengthen firewall rules with a default-deny policy
Although a LOG_DROP chain was used, the environment would benefit from more granular rules that explicitly define allowed ports and protocols for each zone.

4. Enable centralised logging and monitoring
Forwarding Apache, system and firewall logs to a central SIEM platform would allow real-time detection of suspicious activity and provide better visibility across the environment.

5. Apply regular patching and system hardening
Keeping the web server, gateway and workstation updated reduces exploitation risk. Additional hardening measures such as disabling unused services, enforcing SSH key authentication and limiting user privileges improves overall security.

6. Consider brute-force detection and rate limiting on login pages
Adding tools such as Fail2Ban or Apache mod_security can help detect repeated failed login attempts and block attackers automatically.

These recommendations support stronger defence-in-depth and align with best practices for maintaining a secure segmented network environment.

Conclusion

This project successfully demonstrated how a mini SOC environment operates and how network segmentation, firewall filtering and traffic inspection contribute to secure

system design. By configuring routing, NAT, controlled firewall policies and a DMZ web server, the environment reflected real-world network architecture used in enterprise security.

The log analysis and Wireshark captures clearly showed how insecure communication methods expose sensitive information and how proper monitoring can detect such weaknesses. Completing this project strengthened practical skills in Linux administration, firewall management, HTTP analysis and SOC-level investigation. Overall, it provides a strong foundation for further studies and professional development in cybersecurity.